



Le pipeline d'entraînement & la spécialisation

Objectifs du module

Expliquer avec ses mots ce qui se passe « à l'intérieur » d'un LLM au moment de l'inférence, sans maths.

Situer les trois grandes étapes de l'entraînement — pré-entraînement, SFT, alignement — et le rôle de chacune.

Expliquer pourquoi certains modèles sont généralistes et d'autres spécialisés, et comment on obtient cette spécialisation.

Ce module reste conceptuel : on vise l'intuition, pas la formule mathématique de l'attention. Le vocabulaire posé ici sera réutilisé toute la journée — et les six prochains mois.

Plan du module

01	Les bases d'un LLM	~20 min
02	Le cycle de vie d'un LLM	~25 min
03	Généralistes vs spécialisés — le cas du code	~15 min

01

Les bases d'un LLM

Tokens, embeddings, attention, couches — le vocabulaire de base.

**Token après token, un LLM ne fait qu'une chose :
estimer la probabilité du mot suivant.**

Attention, couches, embeddings — tout le reste ne sert qu'à calculer cette probabilité le mieux possible.

Tokenisation et vocab size

Le texte est découpé en **tokens** (sous-mots) — ni lettre par lettre, ni mot par mot. « Anticonstitutionnellement » $\approx$ 4-5 tokens, « chat » $\approx$ 1 token.

Vocab size = nombre de tokens différents connus du modèle (souvent 32k à 200k).

Vocabulaire plus grand $\rightarrow$ tokens plus longs en moyenne $\rightarrow$ séquences plus courtes, mais table d'embedding plus grosse.

[Tokenizer OpenAI](#)

[Tokenizer Huggingface](#)

The screenshot shows the OpenAI tokenizer interface for GPT-5.x & O1/3. It displays the tokenization of the sentence: "Le texte est découpé en tokens (sous-mots) — ni lettre par lettre, ni mot par mot. « Anticonstitutionnellement » $\approx$ 4-5 tokens, « chat » $\approx$ 1 token." The interface shows 49 tokens and 146 characters. Below the text, the tokens are visualized as colored blocks, and the token IDs are shown as numbers. The interface also includes buttons for "Clear", "Show example", "Text", and "Token IDs".

Tokens	Characters
49	146

Deux familles d'algorithmes de tokenisation

Rule-based / whitespace

Découpe simple, sur les espaces et la ponctuation, ou par règles fixes (regex).

Rapide et lisible — mais un mot inconnu (« cryptomonnaie ») reste un token géant, ou tombe hors-vocabulaire. Vocabulaire énorme et rigide : quasi un token par mot possible de la langue.

Plus aucun mot n'est « hors-vocabulaire » avec le BPE : au pire, un mot inconnu se découpe jusqu'aux caractères.

BPE (Byte Pair Encoding)

Algorithme statistique : on part des caractères, puis on fusionne itérativement les paires de symboles les plus fréquentes dans un grand corpus.

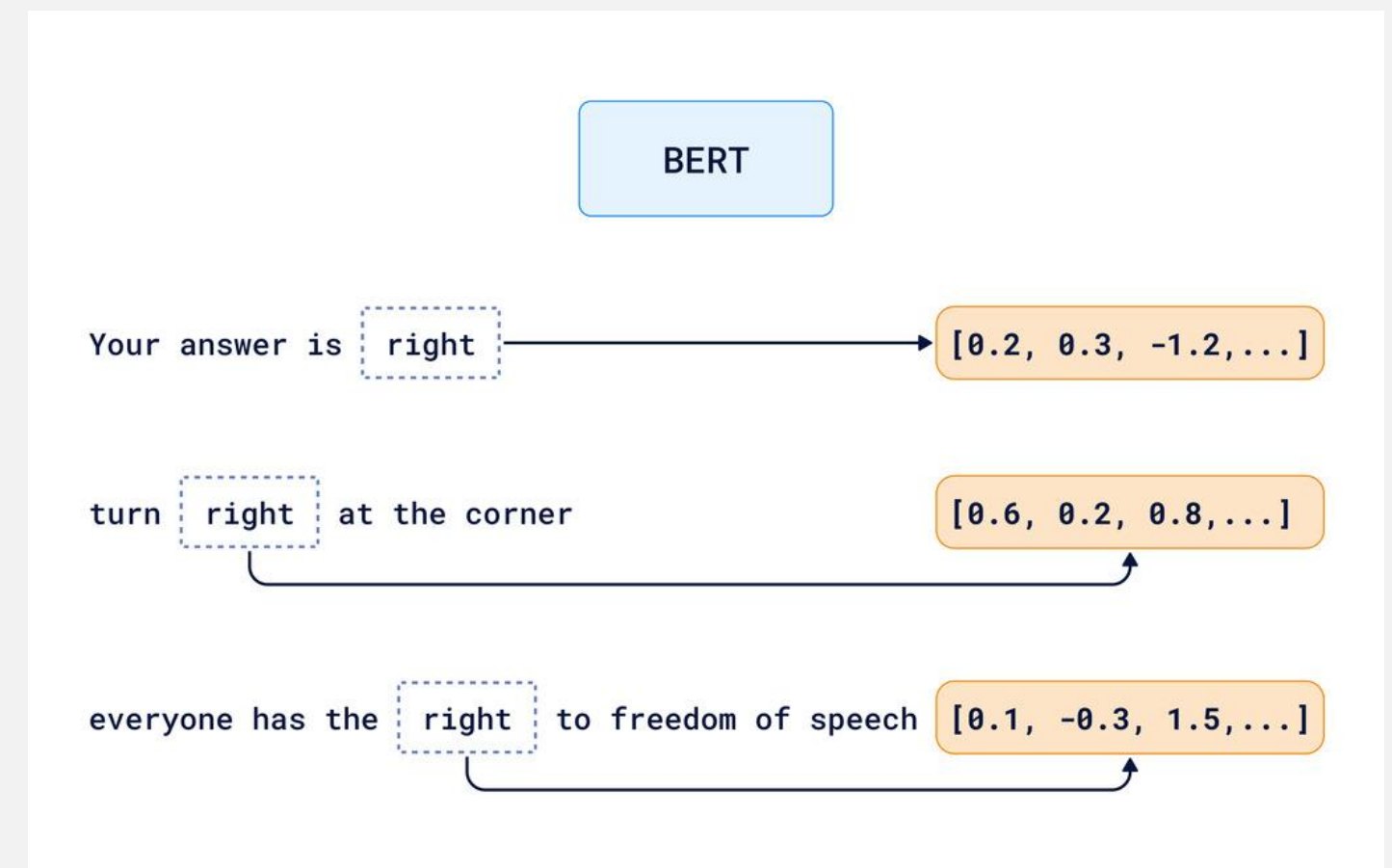
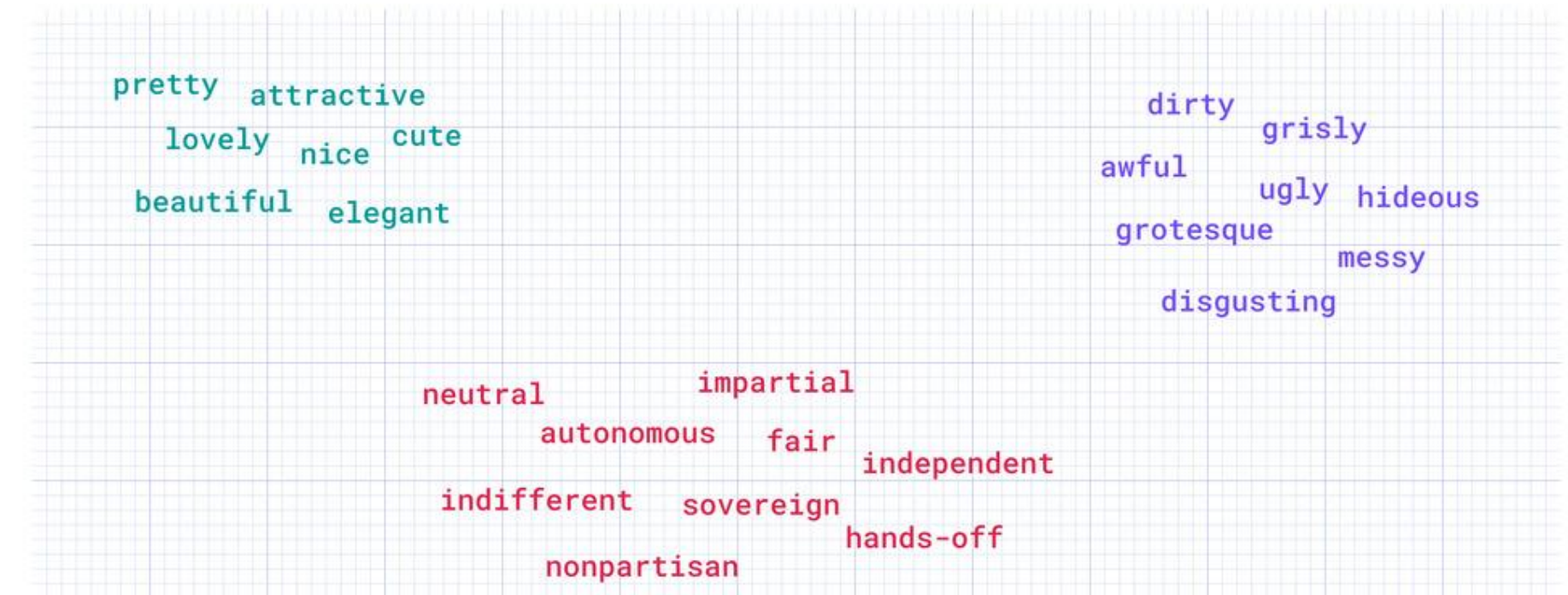
Résultat : un vocabulaire de sous-mots appris automatiquement — les mots fréquents restent entiers, les mots rares se décomposent en morceaux réutilisables. Utilisé (ou proche, ex. SentencePiece) par la quasi-totalité des LLM modernes.

Embeddings : les coordonnées du sens

Chaque token devient un **vecteur** (ex. 4096 dimensions) qui représente son sens.

Comme des coordonnées GPS du sens : « roi » et « reine » sont voisins, « roi » et « banane » sont loin.

Dans un LLM moderne, cet embedding est **contextuel** : le vecteur d'« avocat » change selon qu'on parle du fruit ou du métier — contrairement aux anciens embeddings statiques (Word2Vec).



Le Bag-of-Words : compter les mots

Méthode historique la plus simple : on compte les occurrences de chaque mot du vocabulaire dans le texte, sans se soucier de l'ordre ni de la grammaire. Chaque phrase devient une ligne de chiffres — un vecteur — où chaque colonne correspond à un mot possible du vocabulaire.

PHRASE	LE	CHAT	CHIEN	DORT	AIME
« Le chat dort »	1	1	0	1	0
« Le chien aime le chat »	2	1	1	0	1

Limite fondamentale : « chat » et « félin » sont deux colonnes totalement indépendantes. Le Bag-of-Words ne connaît que des mots, jamais leur sens.

Word2Vec : apprendre le sens par le contexte

Principe : un modèle s'entraîne sur des millions de phrases à prédire un mot à partir des mots qui l'entourent (son contexte).

Deux mots qui apparaissent souvent dans les mêmes contextes reçoivent des vecteurs proches — même s'ils ne sont **jamais** employés côte à côte.

Exemple : « chat » et « félin » se retrouvent tous deux près de « ronronne », « poils », « animal domestique » → vecteurs voisins, malgré des lettres totalement différentes.

Conséquence célèbre : les relations entre mots deviennent des directions dans l'espace — **roi – homme + femme $\approx$ reine** — sans qu'aucune règle de grammaire n'ait été programmée.

Bag-of-Words vs Word2Vec

	BAG-OF-WORDS	WORD2VEC
Unité comptée	Présence/compte d'un mot exact	Contexte d'usage d'un mot
« chat » vs « félin »	Sans rapport	Vecteurs proches
Ordre des mots	Ignoré	Pris en compte pendant l'entraînement
Taille du vecteur	= taille du vocabulaire (souvent > 50 000)	Fixe et compacte (ex. 300 dimensions)

Les embeddings modernes utilisés en RAG (Module 4) reprennent exactement le principe de Word2Vec — « le sens capturé par le contexte » — mais appliqué à des phrases ou des chunks entiers plutôt qu'à un seul mot.

Et aujourd'hui : un embedder moderne

Les embedders actuels (ex. **text-embedding-3** d'OpenAI, ou les modèles open source de la famille **BGE** / **E5**) sont des réseaux de neurones profonds — héritiers directs de Word2Vec, mais entraînés sur des phrases entières plutôt que sur des mots isolés.

Ils produisent un vecteur unique par phrase ou par chunk (souvent 1024 ou 1536 dimensions), qui capture le sens global du passage, contexte grammatical inclus.

Résultat concret : « Comment annuler mon abonnement ? » et « Procédure de résiliation » obtiennent des vecteurs très proches, malgré aucun mot commun — exactement le comportement vu au Module 4 pour la recherche sémantique.

L'attention : pourquoi regarder le contexte

Avant l'attention, un modèle devinait l'importance des mots surtout via leur position ou un résumé compressé de tout ce qui précède.

Principe général : pour interpréter un élément, le modèle calcule un poids d'importance vers chaque autre élément du contexte, puis combine l'information pondérée par ces poids.

Introduite d'abord pour la traduction automatique — aligner un mot traduit avec les mots source pertinents — avant de devenir le cœur des Transformers.

Elle résout le problème de la distance : un lien entre deux mots éloignés dans la phrase se calcule aussi facilement qu'entre deux mots voisins.

Self-attention = un cas particulier : l'attention appliquée à l'intérieur d'une même séquence, chaque token regardant tous les autres tokens de sa propre phrase.

Self-attention : Query, Key, Value

Pour chaque token, le modèle regarde tous les autres tokens de la phrase et pondère leur importance — un moteur de recherche interne.

Query

Ce que le token cherche : « de quoi ai-je besoin pour me comprendre ? »

Key

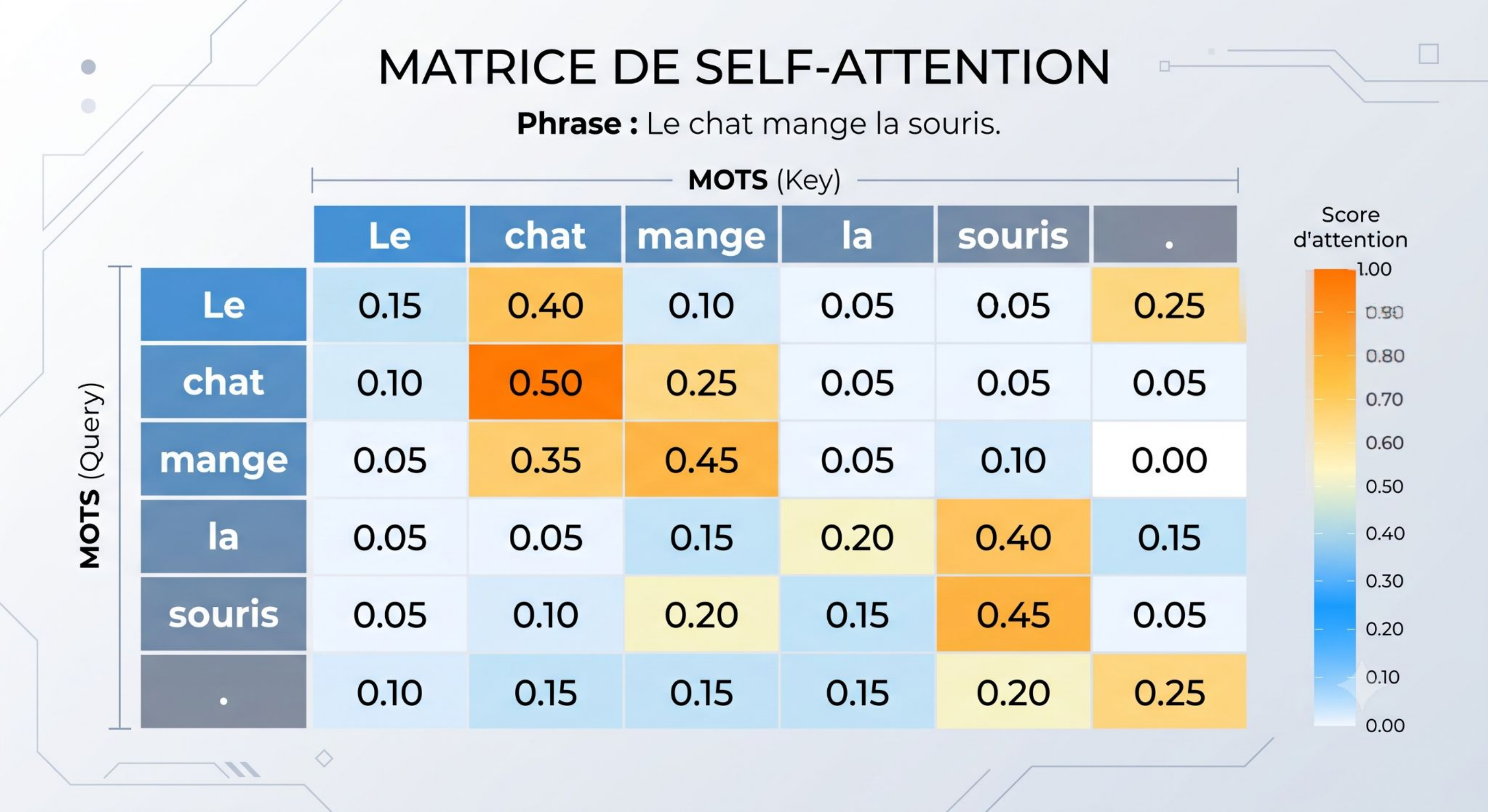
Ce que chaque autre token propose comme information disponible.

Value

Le contenu réellement transmis si le match Query-Key est bon.

Exemple : dans « Le chat qui dort sur le canapé bleu ne bouge plus », interpréter « il » plus loin demande de regarder en arrière vers « chat ». — On ne détaille pas les matrices $W_q/W_k/W_v$: l'objectif est l'intuition, pas le calcul.



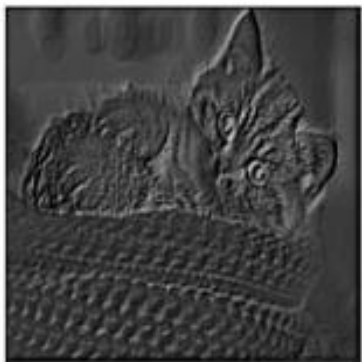
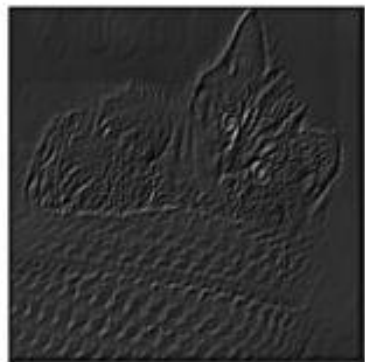
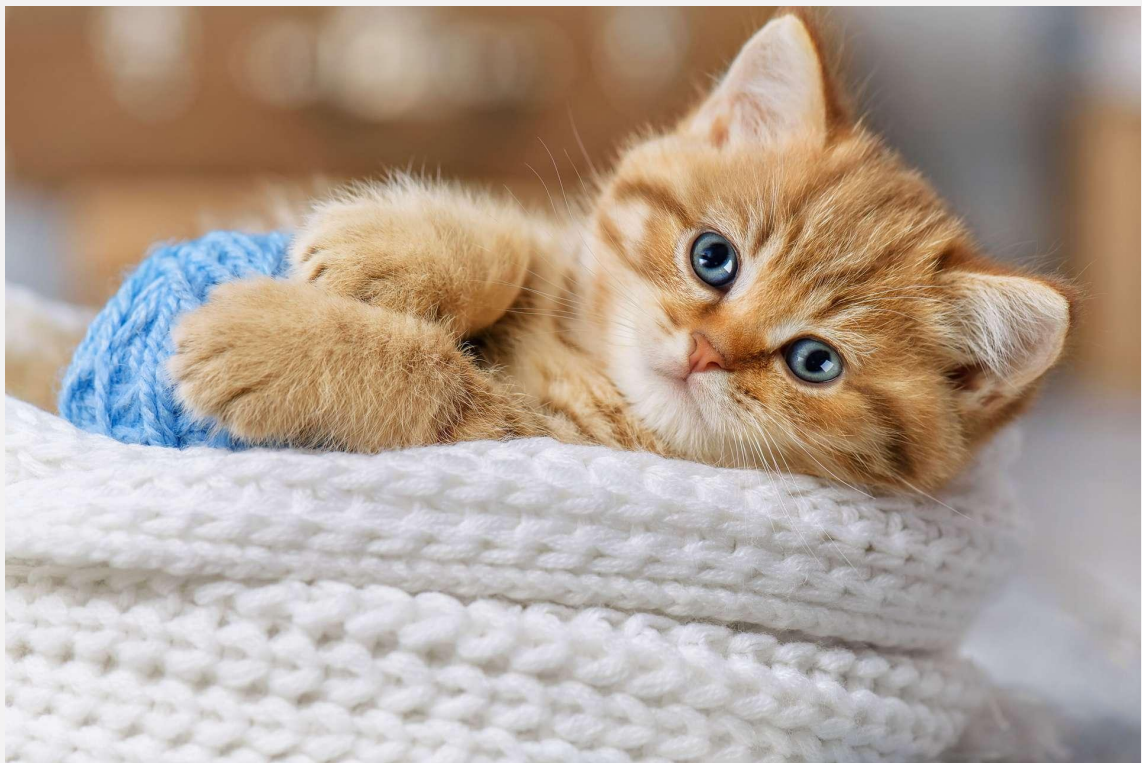


Multi-head attention

Le calcul d'attention est répété plusieurs fois en parallèle, dans des « têtes » différentes.

Chaque tête peut se spécialiser — grammaire, relations de sens, coréférence...

Les résultats des différentes têtes sont ensuite combinés.

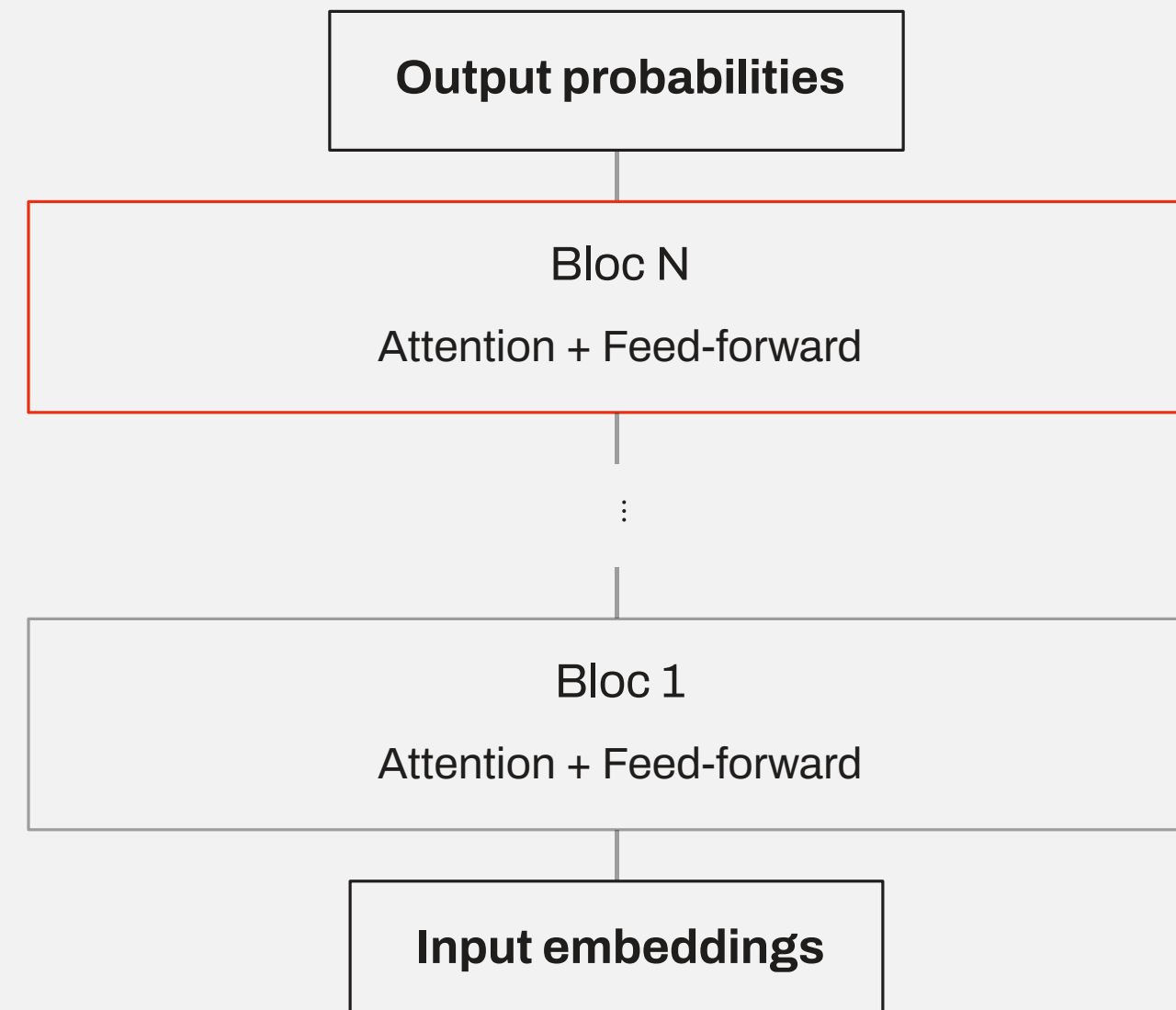


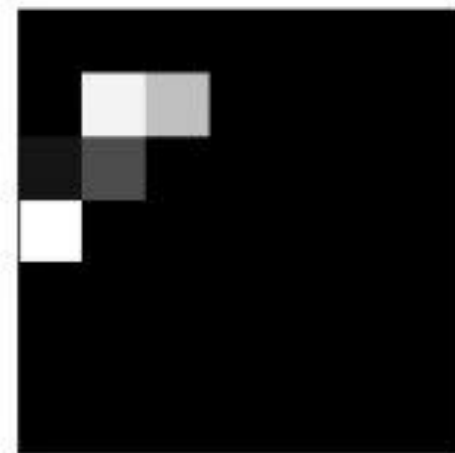
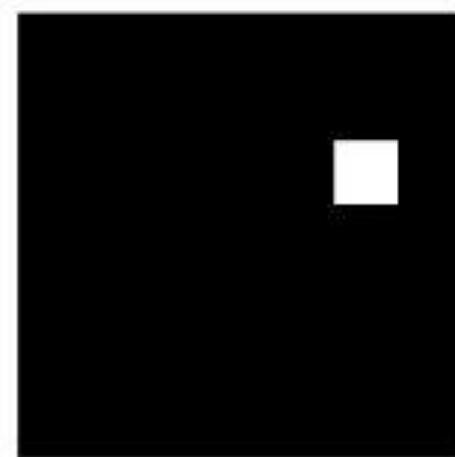
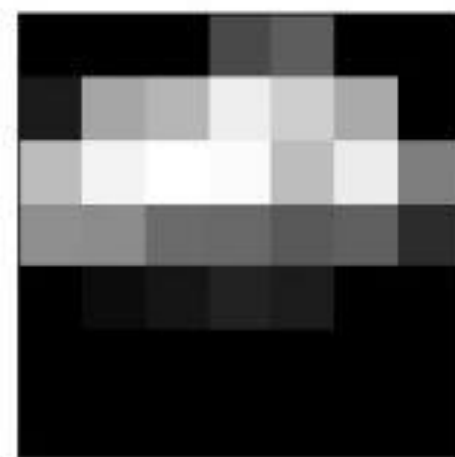
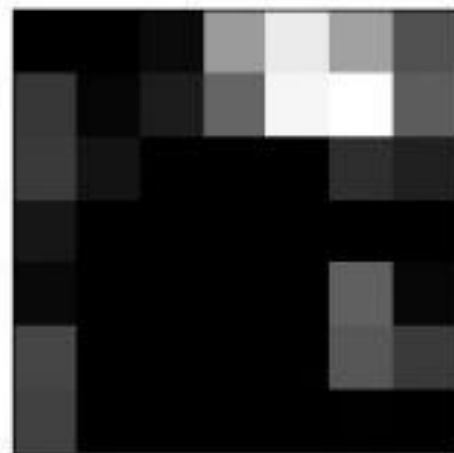
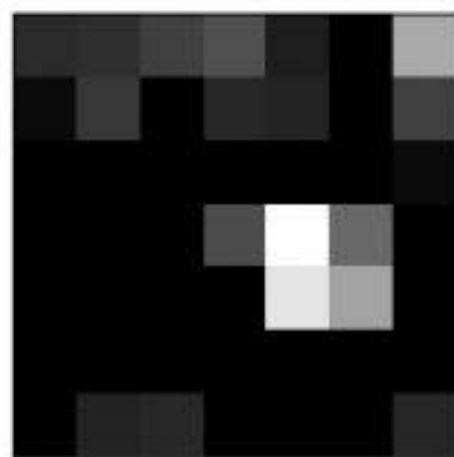
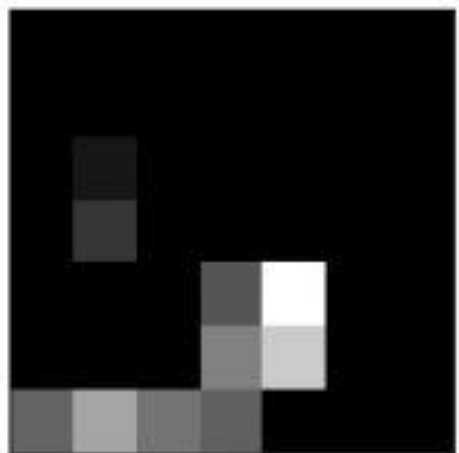
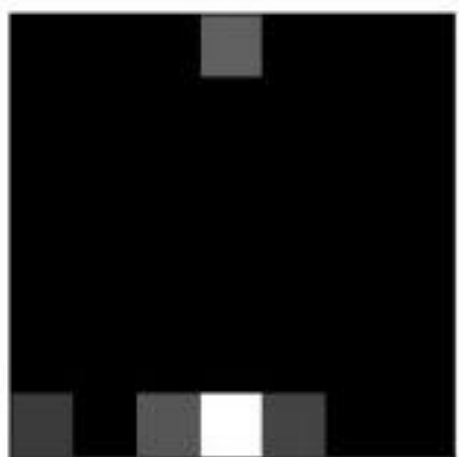
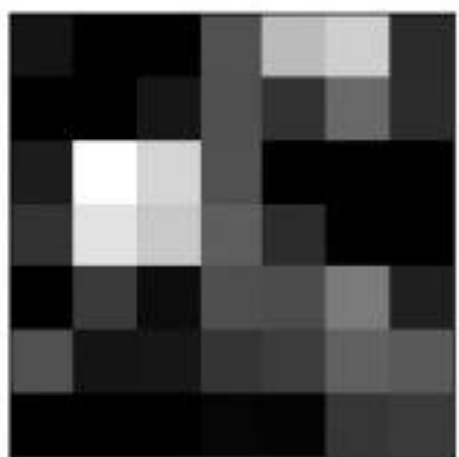
Empilement de couches et paramètres

Un LLM empile des dizaines à centaines de blocs identiques (attention + feed-forward), chacun raffinant un peu plus la représentation.

Chaque couche ajoute un niveau de lecture : syntaxe en surface, relations abstraites en profondeur (logique, style, intention).

Les poids ajustables de ces couches sont les **paramètres** — 7B, 70B, 400B = milliards de paramètres.





02

Le cycle de vie d'un LLM

Pré-entraînement, SFT, alignement — comment un modèle devient un assistant.

Trois étapes, un modèle



Pré-entraînement

« Lire tout internet » — connaissance statistique brute du monde et du langage.



SFT

« Apprendre à discuter » — suivre des instructions, répondre en format conversationnel.



Alignement

« Apprendre les bonnes manières » — sécurité, style, rigueur.

- RLHF
- DPO
- RLVR

Ces trois étapes ne pèsent pas le même poids en données et en calcul : le pré-entraînement en concentre l'écrasante majorité ; SFT et alignement sont beaucoup plus ciblés — mais déterminants pour l'expérience utilisateur finale.

Étape 1 — Pré-entraînement

Apprentissage sur d'énormes volumes de texte brut (web, livres, code, articles...) — souvent plusieurs **trillions de tokens**.

Tâche unique, répétée des milliards de fois : prédire le mot suivant.

Résultat : un **base model** (« modèle de fondation ») — large connaissance du monde, mais qui ne sait pas encore « discuter ».

Coût : l'étape de loin la plus chère — dizaines à centaines de millions de dollars, des mois de calcul sur des clusters de GPU.

Étape 2 — Supervised Fine-Tuning

On ré-entraîne le base model sur des exemples de conversations de qualité : question → bonne réponse, instruction → exécution correcte.

C'est ce qui transforme un « autocomplete de texte » en assistant qui suit des instructions et répond en format chat.

Ces données sont souvent écrites ou curées par des humains, ou générées puis filtrées par d'autres modèles.

Étape 3 — Alignement : RLHF et DPO

Affiner des critères qu'on ne peut pas « montrer par l'exemple » : sécurité, ton, refus appropriés, style, utilité perçue.

RLHF (*Reinforcement Learning from Human Feedback*)

Des humains classent plusieurs réponses du modèle, de la meilleure à la pire. Un « modèle de récompense » apprend ces préférences, puis le LLM est ajusté pour la maximiser.

DPO (*Direct Preference Optimization*)

Une méthode plus récente et plus simple : elle optimise directement à partir des paires « réponse préférée / réponse rejetée », sans entraîner de modèle de récompense séparé.

Exemple de [dataset](#)

RLVR : la récompense vérifiable

Au lieu d'un jugement humain ou d'un modèle de récompense — subjectif — on utilise un vérificateur automatique et objectif.

Un test unitaire qui passe ou échoue, un résultat mathématique exact, une preuve formelle validée.

Explique les gros progrès récents en raisonnement (maths, code) — popularisée notamment par DeepSeek-R1.

Très efficace quand la réussite est vérifiable mécaniquement ; moins bien adapté aux tâches ouvertes (écriture créative, jugement subjectif).

Reinforcement Learning with Verifiable Rewards

Distillation

Un modèle plus petit (« student ») est entraîné à imiter un modèle plus grand (« teacher »), souvent en apprenant directement sur ses réponses et probabilités plutôt que sur des données brutes.

Intérêt : un modèle beaucoup plus petit et moins cher à faire tourner, qui conserve une grande partie des capacités du grand modèle sur les tâches ciblées.

Exemple : les variantes distillées de DeepSeek-R1 (basées sur Qwen/Llama) héritent de ses capacités de raisonnement.

C'est aussi un sujet de tension commerciale et juridique — accusations réciproques entre labs sur l'usage non autorisé des outputs d'un concurrent pour distiller.

Exemple de [dataset](#)

03

Généralistes vs spécialisés

Le cas du code — comment on obtient un modèle expert.

Spécialiser un modèle : le cas du code

Un modèle généraliste doit être bon en moyenne sur énormément de tâches. Un modèle spécialisé sacrifie une partie de cette polyvalence pour exceller sur un domaine précis.

Sur-représentation

Beaucoup plus de code, de dépôts GitHub, de documentation technique dans les données d'entraînement.

Validation automatique

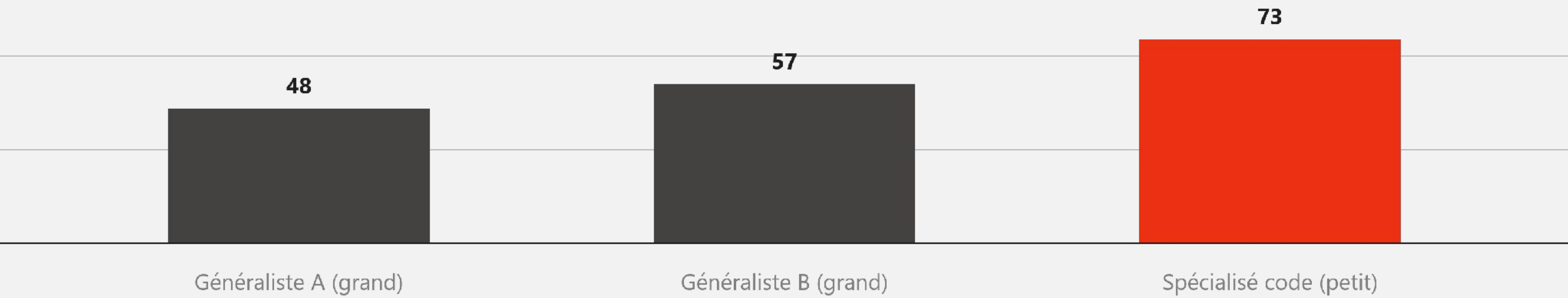
Le code peut être vérifié objectivement — ça compile ou pas, les tests passent ou pas. Un terrain idéal pour le RLVR.

Données synthétiques

On génère massivement du code, on l'exécute, on garde ce qui fonctionne comme nouvelles données — une boucle qui s'auto-alimente.

Le principe se généralise à d'autres domaines vérifiables : mathématiques, biologie/protéines, agents juridiques sur jurisprudence...

Un spécialiste peut battre un généraliste plus gros



Score illustratif sur un benchmark de code (ex. type SWE-bench/HumanEval) — données à vérifier et actualiser juste avant la formation, ce classement évolue vite.

Architectures : dense ou Mixture-of-Experts

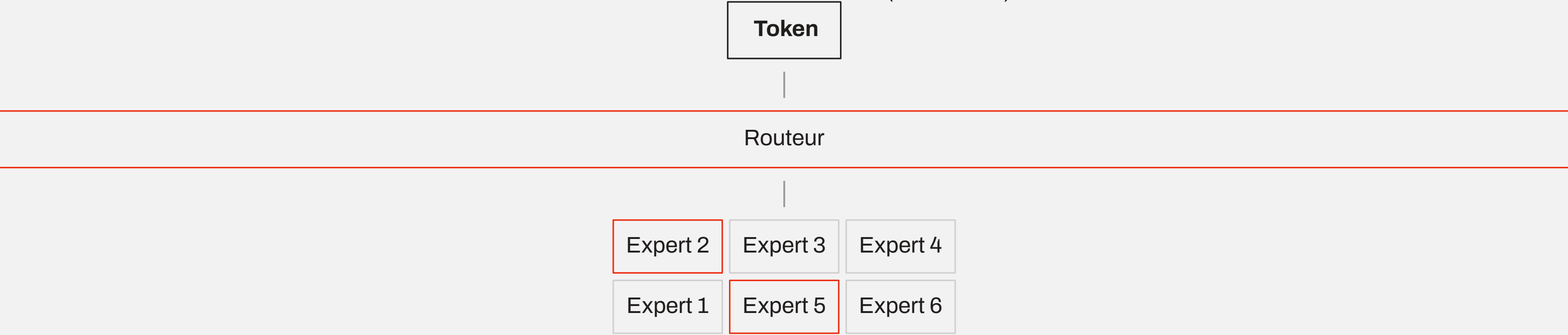
Le pipeline vu jusqu'ici suppose un modèle dense : chaque token active tous les paramètres. Ce n'est pas la seule option.

Dense

Chaque token traverse l'intégralité des blocs et de leurs paramètres. Simple, mais coûteux : plus de paramètres = plus de calcul à chaque inférence.

Mixture-of-Experts

Le bloc feed-forward est remplacé par plusieurs « experts » spécialisés. Un routeur sélectionne, pour chaque token, un petit nombre d'experts à activer (ex. 2 sur 8).



Résultat : un très grand nombre de paramètres totaux, mais un coût de calcul par token proche d'un modèle dense bien plus petit — on gagne en capacité sans payer le calcul intégral. Exemples : Mixtral, DeepSeek-V3.

Synthèse

Architecture

Tokens → embeddings → attention (multi-tête) → empilement de couches → prédiction du token suivant.

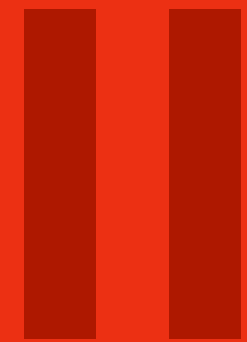
Cycle de vie

Pré-entraînement (connaissance brute) → SFT (format conversationnel) → Alignement RLHF/DPO/RLVR (sécurité, style, rigueur), avec la distillation en option de compression.

Spécialisation

Sur-représentation des données + vérifiabilité automatique (compilateur, tests) = la recette pour battre les généralistes sur un domaine précis.

Maintenant qu'on sait comment un modèle est fabriqué, on va voir qui fabrique quoi — et avec quel écosystème autour.



Le panorama & l'écosystème

Objectifs du module

Nommer les principaux acteurs propriétaires et open weight, et situer la différence entre les deux catégories.

Comprendre le rôle de Hugging Face, des moteurs d'inférence (vLLM, Ollama) et des bases vectorielles dans la chaîne de valeur.

Expliquer les grands ordres de grandeur économiques (coût API vs coût infra) et le compromis vitesse/taille.

Plan du module

01	Les acteurs clés	~25 min
02	Les hubs et l'infrastructure	~25 min
03	Économie & latence	~25 min

01

Les acteurs clés

Propriétaire vs open weight — et la photo début 2026.

Propriétaire vs open weight

Propriétaire (closed / API-only)

Le modèle n'est accessible que via une API payante. On ne connaît ni les poids, ni — souvent — le détail des données d'entraînement.

Acteurs : OpenAI (GPT), Anthropic (Claude), Google (Gemini).

Open weight

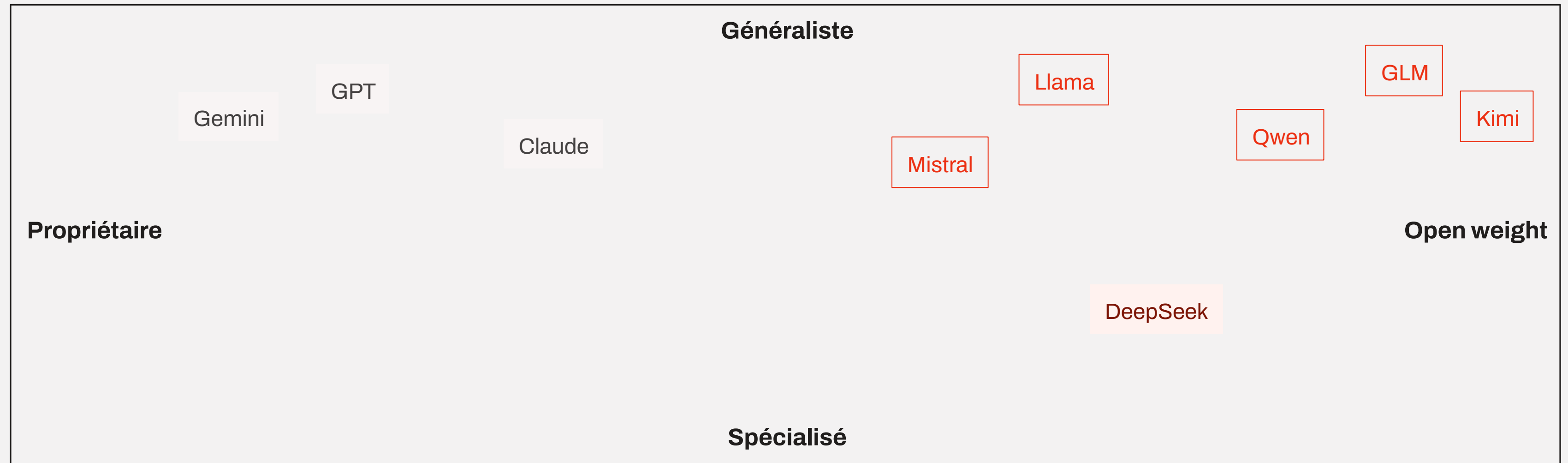
Les poids du modèle sont publiés et téléchargeables (souvent sous licence permissive) — on peut les faire tourner sur sa propre infrastructure.

Acteurs : Meta (Llama), Mistral, Alibaba (Qwen), DeepSeek, Zhipu/Z.ai (GLM), Moonshot AI (Kimi), NVIDIA (Nemotron).

« Open weight » ≠ « open source » au sens classique du logiciel.

La plupart de ces modèles publient les poids — le résultat entraîné — mais pas les données d'entraînement ni le pipeline complet. Un modèle réellement ouvert (poids + données + code d'entraînement) reste rare.

Où en est-on courant 2026 ? (repères)



À redessiner juste avant chaque session avec les familles les plus pertinentes du moment — présenter des « familles évolutives » (« la famille Qwen ») plutôt qu'un modèle figé. Repère : les meilleurs open weight accusent en moyenne environ quatre mois de retard sur la frontière propriétaire (Epoch AI) — un écart qui se resserre, porté par une forte accélération des acteurs chinois (DeepSeek, GLM, Qwen).

02

Les hubs et l'infrastructure

Hugging Face, moteurs d'inférence, bases vectorielles.

Hugging Face — « le GitHub des modèles »

Plateforme centrale de l'écosystème open weight : hébergement de modèles, de datasets, et d'espaces de démonstration (« Spaces »).

Rôle proche de GitHub pour le code : versionning, cartes de modèle (« model cards ») documentant licence, usage et limites, communauté qui contribue des variantes fine-tunées.

Héberge aussi des **leaderboards** (classements de benchmarks) — une ressource de veille pratique, avec Artificial Analysis et LMArena, à réutiliser tout au long des six mois.

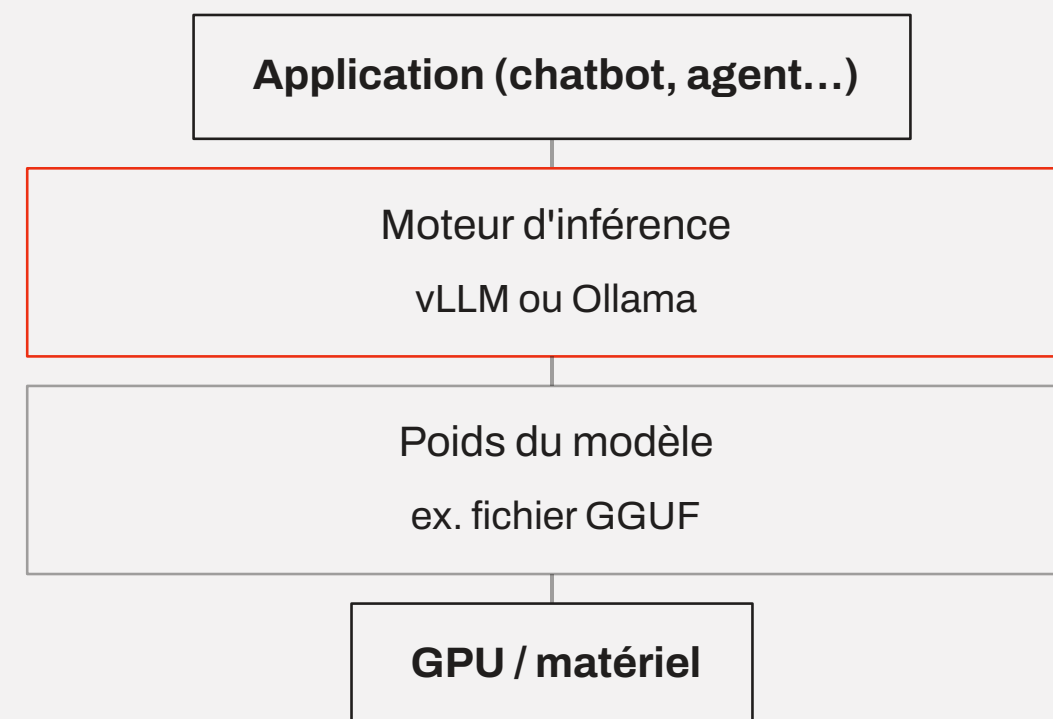
Les moteurs d'inférence : vLLM et Ollama

Un modèle téléchargé (des poids) ne « tourne » pas seul : il faut un **moteur d'inférence** qui les charge en mémoire et gère les requêtes.

vLLM : orienté production/serveur, optimisé pour le débit — servir un modèle à grande échelle en entreprise.

Ollama : orienté usage local/développeur, simple à installer, souvent couplé à des modèles quantifiés (lien direct avec le Module 3).

Analogie : vLLM est à Ollama ce qu'un serveur de production est à un environnement de dev local — même famille d'outils, objectifs différents.



Le stockage sémantique : bases vectorielles

Nécessaires dès qu'on veut chercher par **similarité de sens** plutôt que par mot-clé exact — prérequis direct du RAG (Module 4).

Principe : chaque document est transformé en vecteur (embedding, cf. Module 1), stocké dans une base spécialisée ; on retrouve les vecteurs les plus proches d'une requête.

Outils courants : Pinecone, Weaviate, Milvus, Qdrant, ou des extensions vectorielles de bases classiques (pgvector pour PostgreSQL).

Ce n'est pas un composant « LLM » en soi — c'est une brique d'infrastructure complémentaire, distincte du modèle lui-même.

Fabricants de puces et de mémoire

NVIDIA domine le marché des GPU d'entraînement/inférence (H100, H200, Blackwell) ; AMD (MI300) et les puces maison des hyperscalers (Google TPU, Amazon Trainium/Inferentia, Microsoft Maia) montent en puissance.

La mémoire est un goulot d'étranglement clé : les GPU IA utilisent de la **HBM** (High Bandwidth Memory) — un empilement 3D de puces mémoire au plus près du processeur, pour un débit très supérieur à la RAM classique.

Fabricants de HBM : SK Hynix, Samsung, Micron — un marché concentré sur trois acteurs, souvent plus limitant pour la production de GPU IA que le processeur lui-même.

La disponibilité de HBM conditionne directement le nombre de GPU livrables — donc les délais et les prix de tout l'écosystème.

Pourquoi NVIDIA domine autant : ce n'est pas que le matériel — c'est *CUDA*, sa plateforme logicielle propriétaire. Quinze ans d'outils, de bibliothèques (cuDNN, etc.) et de code écrit par toute l'industrie autour de CUDA créent un verrou logiciel : changer de fournisseur de puces impose de réécrire cet écosystème, pas seulement de changer une carte.

La question de l'énergie

L'entraînement et l'inférence à grande échelle consomment énormément d'électricité — un data center IA moderne peut consommer autant qu'une ville de taille moyenne.

Les grands acteurs investissent directement dans la production d'énergie (contrats nucléaires, renouvelable, parfois redémarrage de centrales) pour sécuriser l'alimentation de leurs data centers.

L'énergie devient un facteur limitant au même titre que les puces — un goulot d'étranglement physique à la croissance de l'IA.

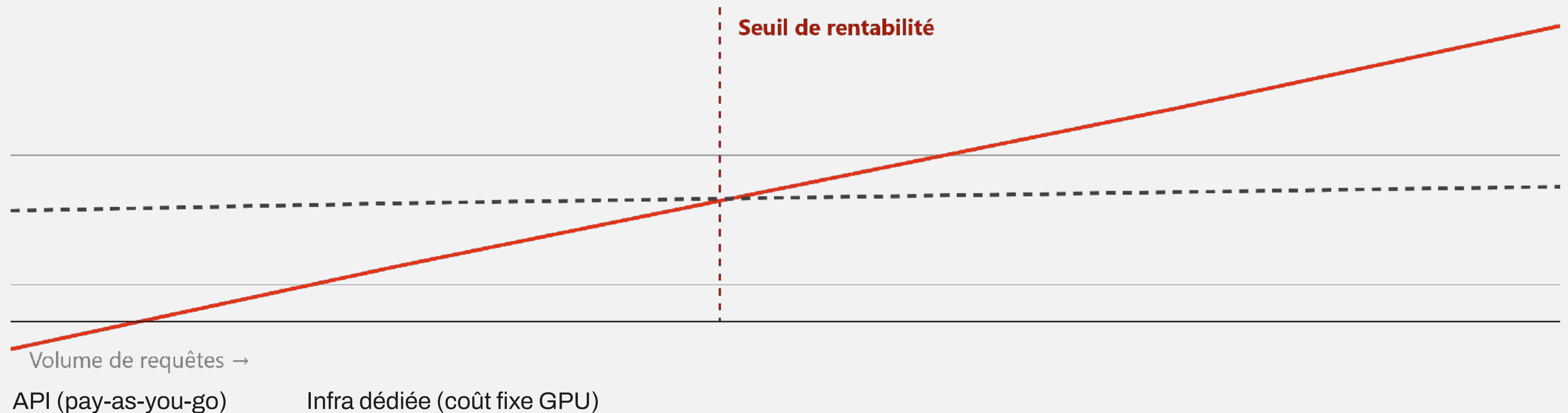
Ressource à montrer en direct : epoch.ai/data/ai-data-centers — cartographie les data centers IA aux États-Unis avec leur puissance et leur capacité.

03

Économie & latence

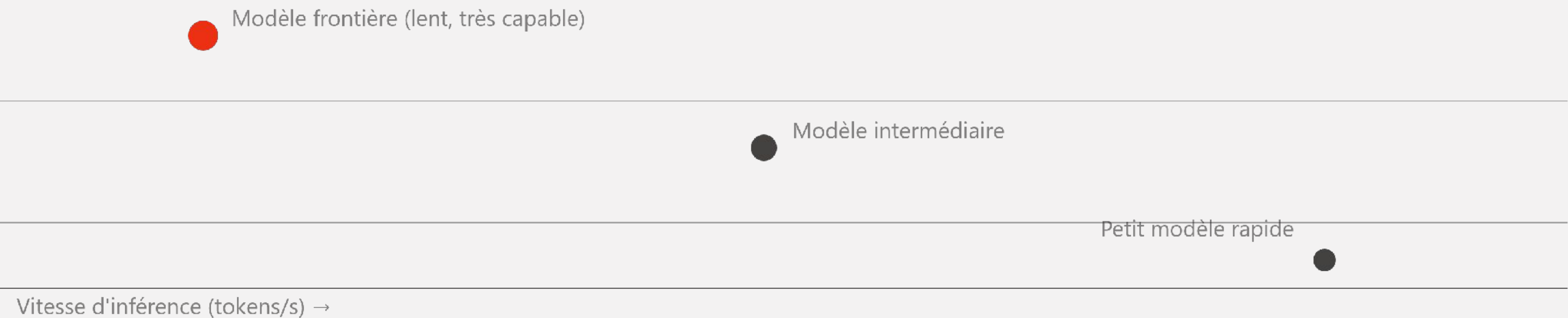
API vs infra dédiée, vitesse vs taille : des arbitrages, pas un meilleur choix universel.

Coût API vs coût d'infrastructure dédiée



À faible volume, l'API est presque toujours plus rentable (pas de coût fixe). Une fois l'infra en place, le coût marginal par requête additionnelle est quasi nul en self-hosted — contrairement à l'API où chaque requête a un coût direct. Le seuil dépend fortement du taux d'utilisation réel du GPU.

Vitesse d'inférence vs taille du modèle



Illustratif — à reconstruire avec des données récentes juste avant la session. Les architectures Mixture-of-Experts (vues au Module 1) permettent de combiner grande capacité totale et inférence plus rapide qu'un modèle dense équivalent, en n'activant qu'une petite partie des paramètres par token.

La latence perçue par l'utilisateur

Un mélange de plusieurs facteurs : taille/architecture du modèle, longueur du prompt, longueur de la réponse générée, charge du serveur, distance réseau.

Un modèle plus petit et plus rapide peut être préférable pour un cas d'usage temps réel (chat interactif), même légèrement moins « intelligent ».

La meilleure réponse n'est pas toujours celle du modèle le plus puissant disponible — c'est un arbitrage, pas un classement absolu.

Synthèse

Le paysage bouge vite

La frontière entre propriétaire et open weight se resserre en continu — l'écart se compte en mois, pas en années.

Un écosystème, pas un modèle

Hubs (Hugging Face), moteurs d'inférence (vLLM/Ollama), bases vectorielles — un projet LLM en production mobilise toute cette chaîne.

Pas de meilleur choix universel

Tout est arbitrage : coût, latence, taille, contrôle.

Cet arbitrage est précisément ce que le Module 3 va creuser : propriétaire vs open source, et la quantification.



Propriétaire vs open source & quantification

Objectifs du module

Argumenter les deux côtés de l'arbitrage propriétaire/open weight avec des critères concrets — pas des opinions.

Expliquer ce qu'est la quantification, pourquoi on la fait, et ce qu'on perd/gagne.

Faire le lien entre les deux sujets : la quantification est souvent ce qui permet en pratique de choisir l'option open weight/self-hosted.

Plan du module

01	Propriétaire vs open source	~35 min
02	La quantification	~35 min

01

Propriétaire vs open source

Un arbitrage business, pas un choix idéologique.

Ce n'est pas les gentils open source contre les méchants propriétaires — c'est un arbitrage business.

On choisit un modèle propriétaire frontière quand le coût d'un échec est élevé et que la capacité maximale prime. On choisit l'open weight quand le contrôle, la confidentialité, la personnalisation ou l'économie à fort volume comptent davantage — la plupart des systèmes sérieux finissent par combiner les deux.

Arguments en faveur du propriétaire

Accès immédiat aux modèles les plus capables du marché, sans investissement matériel.

Maintenance, mises à jour, sécurité et disponibilité (uptime) gérées par le fournisseur.

Facturation à l'usage : pas de coût fixe, adapté aux volumes variables ou en démarrage de projet.

Support et garanties contractuelles (SLA) pour les usages critiques en entreprise.

Arguments en faveur de l'open weight

Souveraineté et confidentialité des données (RGPD) : les données ne quittent jamais l'infrastructure de l'entreprise — central en secteurs réglementés (santé, finance, secteur public), particulièrement sensible en Europe/Belgique.

Pas de dépendance fournisseur (vendor lock-in) : changer de modèle, l'adapter, ne pas dépendre des conditions d'un seul acteur.

Personnalisation profonde : fine-tuning possible sur les poids eux-mêmes — impossible avec un modèle propriétaire fermé.

Coût prévisible à très haut volume , une fois l'infrastructure amortie (lien avec le seuil de rentabilité vu au Module 2).

Les contreparties de l'open weight

Complexité de gestion d'infrastructure : compétences MLOps/infra nécessaires en interne, scalabilité, sécurité du déploiement.

Investissement initial (GPU) : un coût irrécupérable même si le volume d'usage baisse ensuite.

Rester à jour est un travail actif — migrer vers une nouvelle version de modèle est une opération volontaire (tests, re-déploiement), contrairement à une API mise à jour automatiquement par le fournisseur.

Le RGPD (Europe) et l'HIPAA (États-Unis) interdisent souvent strictement la transmission de certaines données sensibles vers des API tierces — un exemple concret qui pousse mécaniquement le secteur santé vers des modèles déployés localement.

Propriétaire ou open weight : le comparatif

CRITÈRE	PROPRIÉTAIRE	OPEN WEIGHT
Coût initial	Aucun	GPU à acheter/louer
Coût à l'usage	Par token, proportionnel	Quasi nul une fois l'infra amortie
Confidentialité	Données transitent chez le fournisseur	Données restent en interne
Contrôle / personnalisation	Prompt seul, fine-tuning limité	Fine-tuning complet sur les poids
Maintenance	Gérée par le fournisseur	À la charge de l'équipe
Accès à la capacité de pointe	Immédiat	Léger retard (~4 mois, cf. Module 2)
Vitesse de mise à jour	Automatique	Migration volontaire

En pratique, beaucoup d'entreprises font un mix : API propriétaire pour les usages non sensibles à forte capacité, open weight self-hosted pour les données sensibles ou le très haut volume — ce n'est pas un camp à choisir définitivement.

02

La quantification

Ce qui rend l'open weight praticable en dehors des géants du cloud.

Pourquoi quantifier ?

Rappel (Module 1) : un modèle est constitué de milliards de paramètres, chacun stocké comme un nombre à virgule flottante — souvent en **FP16** (16 bits, 2 octets) par défaut.

Un modèle de 70 milliards de paramètres en FP16 nécessite environ **140 Go de VRAM** rien que pour charger les poids — inaccessible sur du matériel grand public.

Quantifier = réduire la précision numérique des paramètres pour réduire l'espace mémoire nécessaire, avec une perte de qualité qu'on cherche à minimiser.

Analogie à l'oral : la quantification est une forme de compression avec perte, un peu comme le JPEG pour les photos — on réduit le poids du fichier en acceptant une perte de qualité, en visant le point où elle devient imperceptible pour l'usage réel.

Les formats principaux (1/2)

FP16 / BF16

Précision « pleine » de référence — la plus lourde en mémoire, le point de départ avant toute quantification.

INT8

Réduction de moitié par rapport à FP16 — perte de qualité généralement faible.

Les formats principaux (2/2)

INT4 (AWQ, GPTQ)

Réduction drastique (~4x plus léger que FP16) — permet de faire tourner un modèle 70B sur un seul GPU. AWQ protège les poids les plus importants pour la qualité d'une quantification trop agressive.

GGUF

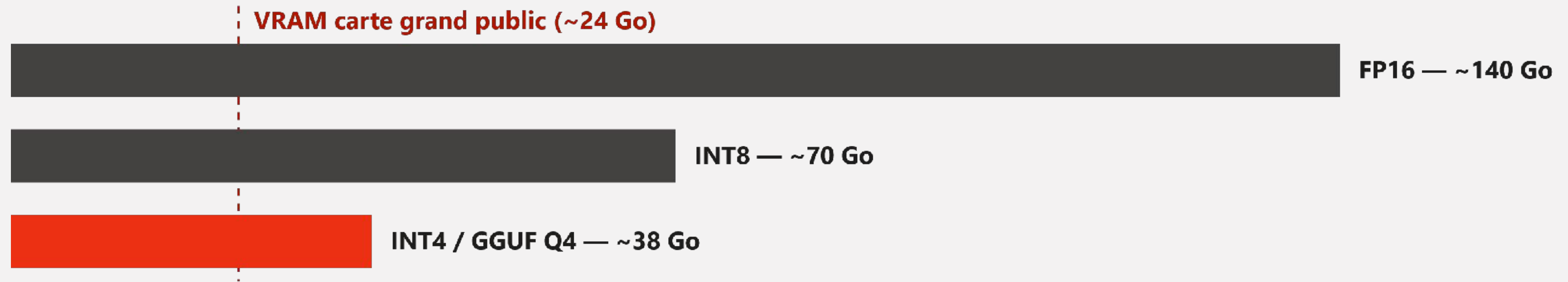
Un format de fichier (pas une technique) très répandu pour l'usage local, notamment via Ollama. Inférence hybride CPU+GPU — le standard de facto pour distribuer des LLM sur du matériel grand public.

FP8

Format récent, pris en charge nativement par les GPU les plus modernes — calcul direct, sans étape de déquantification logicielle contrairement à l'INT4.

L'essentiel à retenir : plus on réduit la précision, plus on gagne en mémoire et en vitesse, plus on risque de perdre en qualité — le bon choix dépend du matériel disponible et de l'usage visé.

Un même modèle 70B, selon le format



Ordres de grandeur pour un modèle de 70 milliards de paramètres — à recalculer si le modèle de démo change (nombre de paramètres × taille en octets du format).

Ordres de grandeur, aujourd'hui

Grand public

1 à 9 milliards de paramètres
(ex. Gemma-4-31B, Gemma-4-12B) —
tournent sur un laptop ou un mobile, souvent
quantifiés.

Intermédiaires

30 à 70+ milliards (ex. Llama 3.1 70B, Qwen
72B) — nécessitent un ou plusieurs GPU
dédiés.

Frontières

Propriétaires (GPT, Claude, Gemini) : tailles
rarement publiées, estimées à plusieurs
centaines de milliards, souvent en MoE. Open
weight géants : DeepSeek-V3 (~671B, MoE),
Llama 405B.

L'impact de la fenêtre de contexte

La **fenêtre de contexte** est le nombre maximum de tokens que le modèle peut traiter en une fois — prompt et réponse cumulés.

Elle a beaucoup grandi : de 8k-32k tokens il y a peu, à 128k voire plus de 1 million de tokens pour les modèles les plus récents.

Plus la fenêtre réellement utilisée est longue, plus la mémoire nécessaire pendant l'inférence augmente — pas seulement pour les poids, mais pour tout l'état interne du calcul en cours (le KV cache, page suivante).

L'impact du KV cache sur la mémoire

Rappel (Module 1) : à chaque token généré, l'attention a besoin des Key/Value de tous les tokens précédents.

Plutôt que les recalculer à chaque nouveau token, le modèle les met en mémoire : c'est le **KV cache**.

Ce cache grandit linéairement avec la longueur du contexte **et** avec le nombre de requêtes traitées en parallèle — il peut devenir plus lourd que les poids eux-mêmes sur des contextes longs ou avec beaucoup d'utilisateurs simultanés.

La mémoire nécessaire pour « servir » un modèle n'est donc jamais juste « poids quantifiés » : il faut prévoir une marge pour le KV cache, selon la fenêtre de contexte et le nombre d'utilisateurs simultanés visés.

Le compromis qualité/mémoire en pratique

Recommandation générale pour la production : privilégier le FP8 si le matériel le supporte, sinon l'AWQ INT4 si l'espace mémoire est la contrainte principale.

En développement local, un format GGUF (type Q4_K_M ou Q5_K_M) reste le choix le plus courant.

Pour une conversation ou un raisonnement généraliste, la perte de qualité d'une quantification 4 bits bien faite se limite souvent à quelques points de pourcentage — la plupart des utilisateurs ne perçoivent pas la différence en aveugle.

Repère pratique : commencer par une quantification 4 bits par défaut, et ne revenir à une précision supérieure que si une dégradation gênante est mesurée sur le cas d'usage réel.

À un niveau trop agressif (ex. 2 bits), la dégradation devient nette et perceptible, notamment sur des tâches de raisonnement complexe — plus de quantification n'est pas toujours mieux.

Bonus — le fine-tuning quantifié (QLoRA)

Combine quantification et fine-tuning : on entraîne un modèle sur du matériel limité en ne mettant à jour qu'une petite partie des poids, sur une base quantifiée.

Sert de pont naturel vers le Module 4 de l'après-midi, où le fine-tuning sera détaillé.

Synthèse

La quantification est la technique qui rend l'arbitrage « open weight » réellement praticable en dehors des géants du cloud.

Ce module clôt la matinée sur le triptyque : comment c'est fait → dans quel écosystème → avec quels compromis économiques/techniques.

L'après-midi bascule sur un terrain différent : comment on utilise concrètement ces modèles dans une application — prompt, RAG, fine-tuning, agents.

IV

**Structurer et augmenter :
Prompt vs RAG vs Fine-Tuning**

Objectifs du module

Expliquer le principe des embeddings et de la recherche sémantique appliqués à la recherche d'information — pas seulement à la compréhension d'un token, vue au Module 1.

Utiliser une grille de décision claire pour choisir entre Prompt Engineering, RAG et Fine-Tuning face à un besoin donné.

Justifier pourquoi, dans la majorité des cas réels, la réponse n'est « ni l'un ni l'autre » mais une combinaison des trois.

Plan du module

01	Embeddings et recherche sémantique	~25 min
02	Prompt vs RAG vs Fine-Tuning	~1h

01

Embeddings et recherche sémantique

Le même principe qu'au Module 1, appliqué à des documents entiers.

Du token au document

Les embeddings ont été introduits au Module 1 au niveau du token. Ici, on applique le même principe à des **documents entiers ou des paragraphes** (« chunks ») plutôt qu'à des mots isolés.

Chaque chunk de texte — un paragraphe de politique interne, un article de documentation, une fiche produit... — est transformé en un vecteur qui représente son sens global.

Le principe de la recherche sémantique

Une recherche par mots-clés classique (Ctrl+F, `LIKE` en SQL) ne trouve que des correspondances exactes de texte.

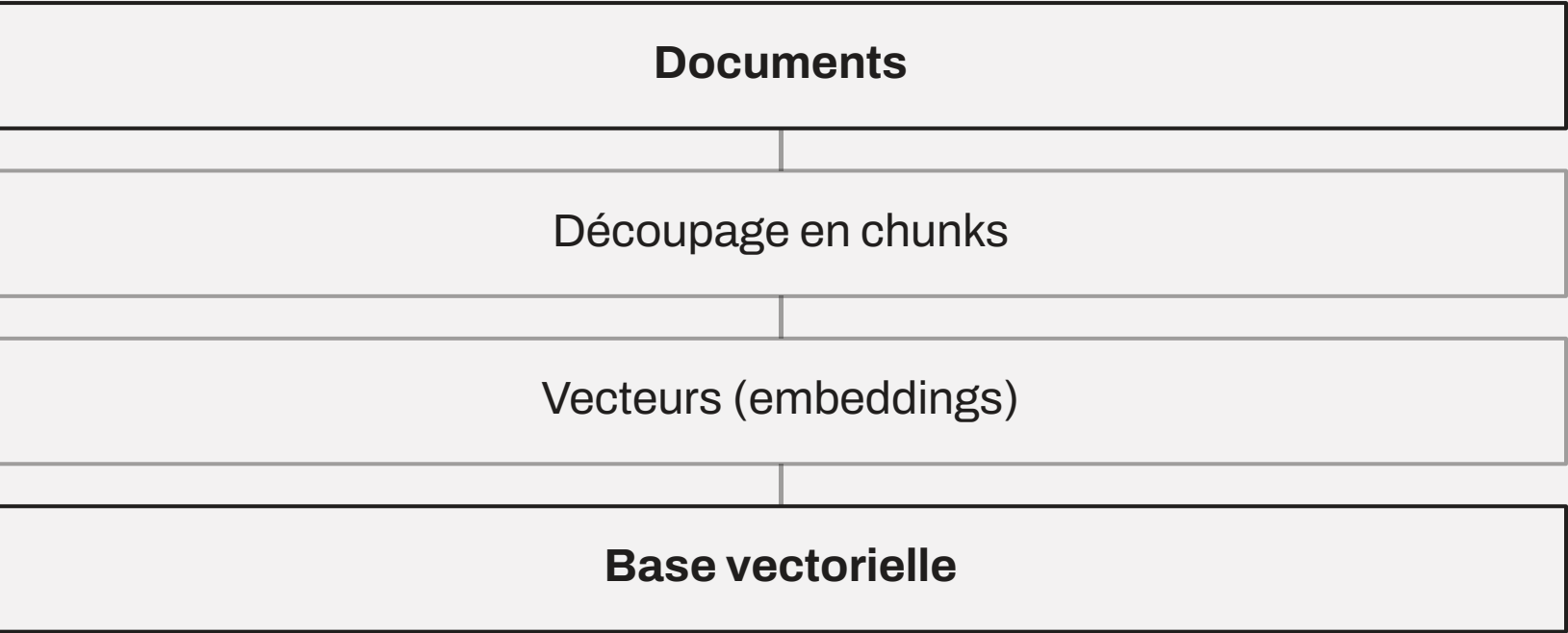
La recherche sémantique compare des **vecteurs** : elle retrouve les chunks dont le sens est le plus proche de la question, même si aucun mot n'est identique.

Exemple : « Comment annuler mon abonnement ? » doit retrouver un paragraphe intitulé « Résiliation et remboursement » — sans le mot « annuler ».

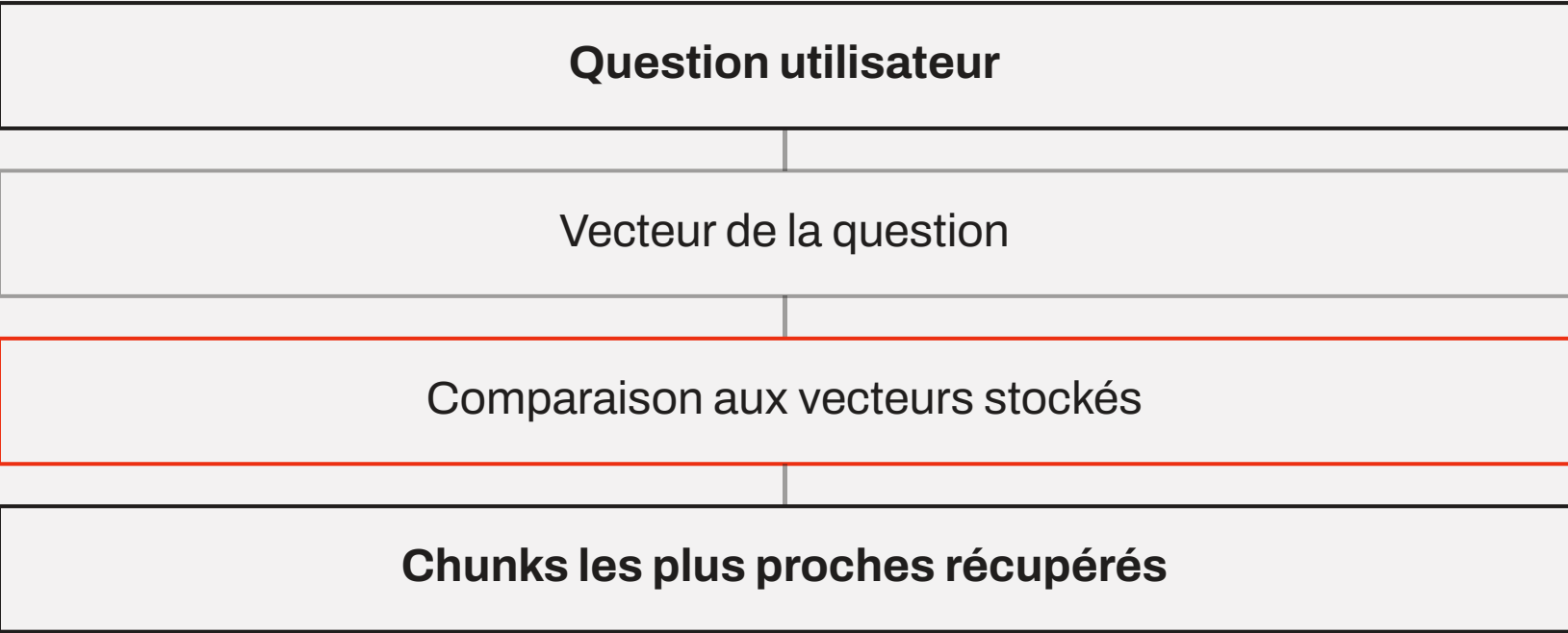
Cette proximité se mesure via une similarité cosinus — l'intuition suffit : plus deux vecteurs pointent dans la même direction dans cet espace de sens, plus les textes sont proches sémantiquement.

De la base de documents à la réponse

1. Indexation (hors ligne)



2. Recherche (à la requête)



Le chunking : un choix qui compte

Le **chunking** — comment on découpe les documents en morceaux avant de les vectoriser — a un impact réel sur la qualité de la recherche.

Trop petit : perte de contexte. Trop grand : bruit et dilution de la pertinence.

Le RAG ne « fonctionne pas tout seul » : la qualité du chunking conditionne la qualité de tout ce qui suit.

02

Prompt vs RAG vs Fine-Tuning

Une grille de décision, pas un choix idéologique.

Trois options, une phrase chacune

Prompt Engineering

On ne change rien au modèle ; on soigne uniquement ce qu'on lui envoie — instructions, exemples, format attendu — pour orienter sa réponse.

RAG

On laisse le modèle intact, mais on lui injecte automatiquement, à chaque requête, des informations récupérées depuis une base externe.

Fine-Tuning

On modifie réellement les poids du modèle (ou une partie, via LoRA/QLoRA) en le ré-entraînant sur des exemples spécifiques — le savoir est « gravé » dans le modèle.

Prompt d'abord, RAG ensuite, Fine-Tuning en dernier recours.

Chaque étape suivante coûte significativement plus cher — temps, argent, compétence — que la précédente. Ce n'est pas un dogme rigide, mais une heuristique de bon sens économique : un pipeline RAG bien construit couvre à lui seul 80 à 90 % des besoins d'applications à forte intensité de connaissance.

Les critères de décision (1/3)

Fraîcheur de l'information

Le RAG donne un accès en temps réel à l'information ; le fine-tuning fige une connaissance statique au moment de l'entraînement. Infos qui changent souvent (stock, actualité, prix) → RAG.

Savoir vs comportement

Le RAG répond bien pour donner accès à des faits ; le fine-tuning est nécessaire pour imposer un ton, une persona ou une terminologie constante — le style est un comportement, pas un fait.

Les critères de décision (2/3)

Traçabilité et auditabilité

Le RAG ancre les réponses dans des documents identifiables — on peut citer la source. Le fine-tuning peut renforcer des hallucinations confiantes, sans traçabilité de la source.

Contrôle d'accès aux données

Le RAG permet un contrôle d'accès fin au niveau de la base — chaque utilisateur ne récupère que ses documents autorisés. Le fine-tuning ajoute l'information dans les poids, accessible à quiconque interroge le modèle.

Les critères de décision (3/3) — le coût

Le RAG déplace la charge vers la mémoire et les accès base de données ; le fine-tuning déplace la charge vers du calcul GPU intensif.

Prompt engineering : ne coûte que du temps (heures/jours).

RAG : coût d'infrastructure continu (base vectorielle, pipeline d'ingestion).

Fine-tuning : investissement en mois de travail, avec un coût d'inférence pouvant être plus élevé qu'un modèle de base.

Quand le fine-tuning reste le meilleur choix

Le cas commercial le plus solide aujourd'hui : distiller un modèle frontière performant vers un petit modèle open weight ajusté, pour diviser le coût d'inférence d'un ordre de grandeur sur une tâche étroite — lien direct avec la distillation vue au Module 1.

Autre cas : un prompt devenu ingérable. S'il faut plusieurs milliers de tokens d'instructions à chaque appel pour que le modèle se comporte correctement, le fine-tuning permet de réduire cela à quelques dizaines de tokens — moins cher et plus rapide à chaque requête.

La matrice de décision

CRITÈRE	PROMPT ENGINEERING	RAG	FINE-TUNING
Coût de mise en œuvre	Faible	Moyen	Élevé
Vitesse de mise en œuvre	Immédiate	Jours à semaines	Semaines à mois
Fraîcheur des données	Limitée au contexte	Temps réel	Figée à l'entraînement
Traçabilité	—	Sources citables	Faible
Sécurité / contrôle d'accès	—	Fin, par document	Aucun (cuit dans le modèle)
Capacité à changer le style/ton	Limitée	Limitée	Forte
Coût d'inférence à long terme	Faible	Moyen	Variable (baisse si distillation)

L'arbre de décision, en pratique

Le modèle a-t-il besoin d'infos qu'il ne connaît pas déjà ?

— Oui

→ RAG

| Non

Le problème est-il un problème de ton/format/comportement répété plutôt qu'un manque de connaissance ?

— Oui

→ Envisager le Fine-Tuning

| Sinon

→ Rester sur le Prompt Engineering

La combinaison, pas l'exclusion

La vraie question en entreprise n'est presque jamais « RAG contre fine-tuning » mais « quel est le bon pipeline pour ce cas d'usage précis » — la plupart des entreprises sérieuses combinent les trois approches selon les besoins.

Schéma le plus courant en production : on fine-tune un petit modèle open weight sur le style ou le raisonnement métier nécessaire, puis on l'enveloppe avec du RAG pour qu'il réponde à partir des connaissances à jour de l'entreprise.

Vigilance finale : la barre pour justifier un fine-tuning est de plus en plus haute — les modèles de base actuels gèrent nativement une grande partie de ce qui nécessitait un ajustement il y a encore 18 mois.

Synthèse

Ordre par défaut : Prompt → RAG → Fine-Tuning, jamais l'inverse sans raison validée.

Les critères qui font pencher la balance : fraîcheur des données, nature du besoin (savoir vs comportement), traçabilité/sécurité, coût.

La réalité de production est quasi toujours hybride.

Le Module 5 va montrer comment un système peut lui-même décider dynamiquement d'aller chercher de l'information — le RAG comme outil d'un agent, plutôt qu'une logique câblée en dur.

V

**Vers l'autonomie : Agents,
Tools & Skills**

Objectifs du module

Expliquer la différence entre un LLM « simple » (texte → texte) et un système agentique qui peut agir sur son environnement.

Décrire la boucle ReAct (raisonnement + action) et le rôle du tool calling dans cette boucle.

Distinguer mémoire court terme (contexte de la conversation) et mémoire long terme (persistante entre les sessions).

Situer où en est l'écosystème des agents aujourd'hui — la standardisation autour du protocole MCP notamment.

Plan du module

01	Anatomie d'un agent	~25 min
02	La boucle ReAct	~35 min
03	L'écosystème des agents aujourd'hui	~25 min

01

Anatomie d'un agent

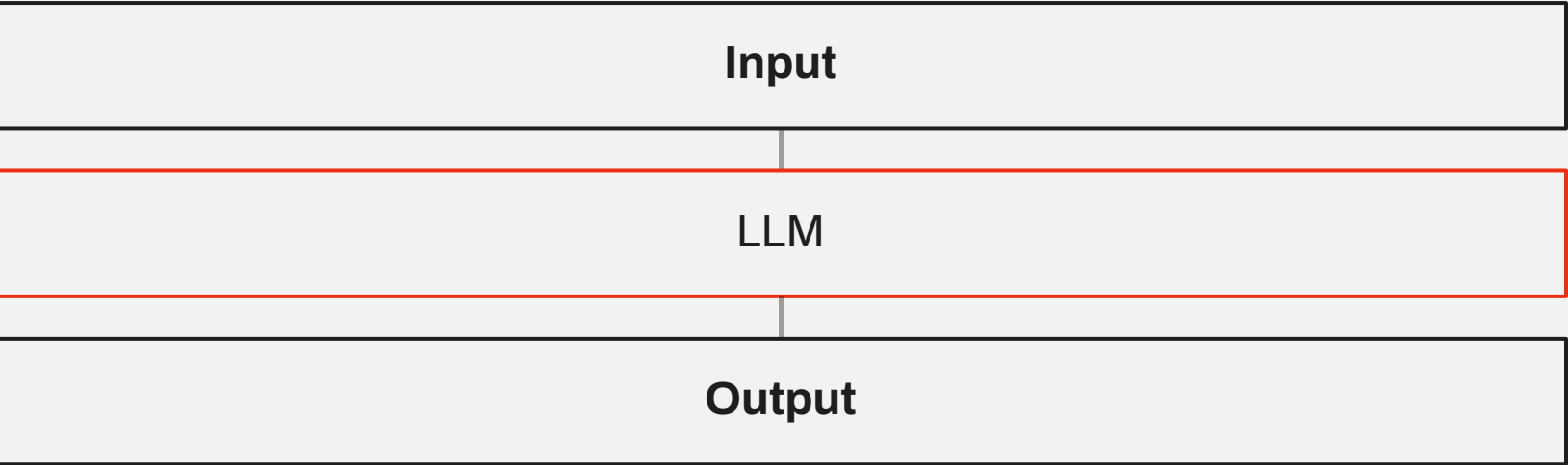
Générateur de texte vs système qui agit.

Un LLM seul est un cerveau sans mains.

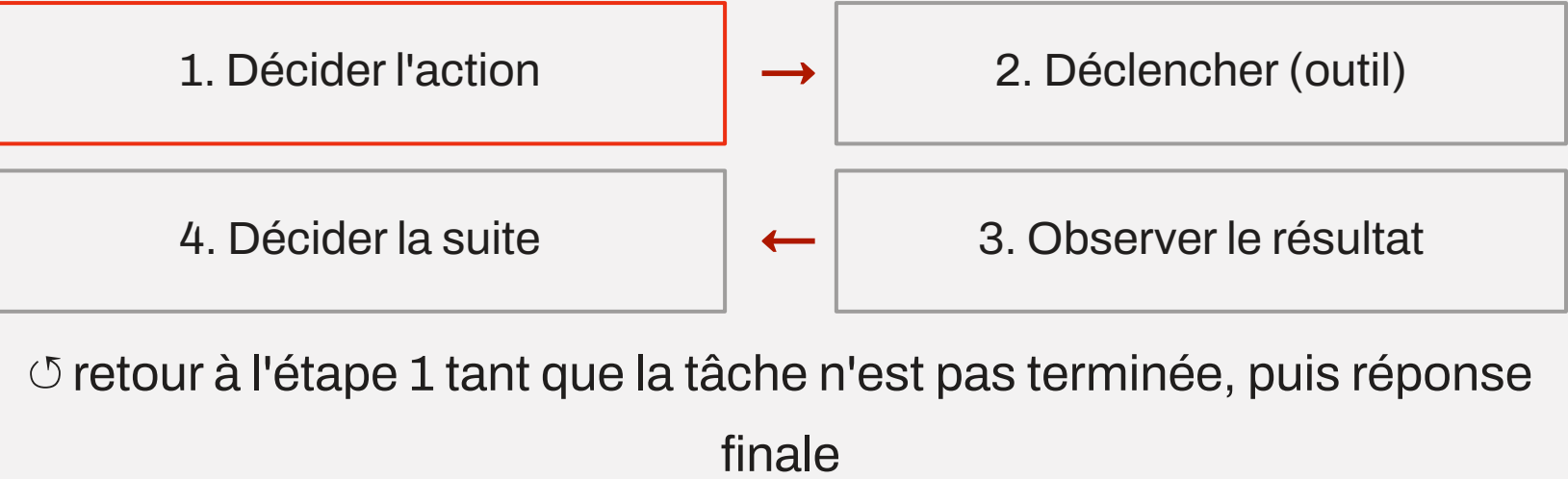
Il peut réfléchir et parler, mais pas agir sur le monde — consulter un site à jour, envoyer un email, interroger une base de données, exécuter du code. Un agent lui donne des mains — des outils — et une boucle de décision pour savoir quand les utiliser.

D'un flux linéaire à une boucle

LLM simple



Système agentique



Ce qui fait la difficulté d'un agent

Ce n'est **pas** la génération de texte elle-même — le LLM sait déjà très bien faire ça.

C'est la fiabilité de la boucle de décision : choisir le bon outil, au bon moment, avec les bons paramètres — et savoir s'arrêter.

C'est là que se concentre la majorité des erreurs en pratique.

02

La boucle ReAct

Reasoning + Action — alterner réflexion et action.

Le principe ReAct

ReAct fait alterner explicitement, dans le raisonnement du modèle, des étapes de **réflexion** (« que dois-je faire maintenant ? ») et des étapes d'**action** (« j'appelle tel outil avec tels paramètres »), suivies d'une **observation** du résultat.

Ce cycle peut se répéter plusieurs fois avant la réponse finale — plusieurs outils appelés successivement, ou un même outil appelé plusieurs fois avec des paramètres différents.

Un exemple concret

Thought	« L'utilisateur demande la météo à Liège aujourd'hui. Je dois utiliser l'outil météo. »
Action	get_weather(ville="Liège")
Observation	« 18°C, nuageux »
Thought	« J'ai l'information nécessaire, je peux répondre. »
Réponse finale	« Il fait 18°C et nuageux à Liège aujourd'hui. »

Tool calling : décider n'est pas exécuter

Tool calling (ou « function calling ») = la capacité du modèle à produire, à la place d'une réponse en texte libre, une structure indiquant « j'appelle telle fonction avec tels paramètres ».

Cette structure est exécutée par le système qui entoure le modèle — jamais par le modèle lui-même.

Le LLM ne « fait » jamais l'action lui-même — il ne fait que décider et formuler l'appel ; c'est le code autour de lui qui exécute réellement l'action et lui renvoie le résultat.

Chain-of-Thought et modèles « raisonneurs »

Le **Chain-of-Thought** fait produire au modèle son raisonnement intermédiaire, étape par étape, avant sa réponse finale — améliore la qualité sur des tâches complexes (maths, logique, planification), au prix d'une réponse plus longue.

Les modèles « raisonneurs » sont entraînés — notamment via RLVR (Module 1) — à produire de longues chaînes de raisonnement avant de répondre.

Plus fiables comme « cerveau » d'un agent complexe, au prix d'une latence et d'un coût par requête plus élevés.

Sans CoT vs avec CoT : un exemple

Sans CoT — réponse directe

« Un fût contient 3 boules rouges et 5 boules bleues. On tire 2 boules sans remise. Quelle est la probabilité que les deux soient rouges ? »

Réponse

« Environ 3/8 » — **faux** , calculée à l'instinct sans poser les tirages successifs.

Avec CoT — raisonnement explicite

Étape 1	$P(1\text{re rouge}) = 3/8$
Étape 2	Sans remise → $P(2\text{e rouge} \mid 1\text{re rouge}) = 2/7$
Résultat	$3/8 \times 2/7 = 6/56 = \mathbf{3/28}$

Le CoT ne rend pas le modèle plus « intelligent » — il force la décomposition, ce qui réduit les erreurs sur les tâches à plusieurs étapes.

Régler l'effort de raisonnement : low / medium / high

Les modèles raisonneurs récents exposent un paramètre (**reasoning effort** ou **thinking budget**) qui règle la longueur autorisée de la chaîne de pensée avant la réponse.

NIVEAU	CE QUI SE PASSE	LATENCE / COÛT	CAS D'USAGE TYPIQUE
Low	Peu ou pas d'étapes intermédiaires, quasi équivalent à une réponse directe	Faibles	Chat courant, classification, reformulation
Medium	Quelques étapes de raisonnement, suffisant pour la plupart des tâches à logique modérée	Modérés	Résumé analytique, code simple, réponse à un ticket support
High	Chaîne de raisonnement longue, exploration de plusieurs pistes avant de conclure	Élevés	Maths avancées, debug complexe, planification multi-étapes d'un agent

Choisir le niveau, c'est arbitrer qualité contre coût et latence — pas besoin de « high » pour une simple reformulation.

Mémoire court terme et long terme

Court terme

Le contexte de la conversation en cours — limité par la fenêtre de contexte du modèle (lien direct avec la tokenisation du Module 1).

Exemple : un assistant qui se souvient, d'une session à l'autre, que l'utilisateur préfère les réponses courtes — ce n'est pas « le modèle qui se souvient » mais un système de mémoire externe qui réinjecte cette information dans le prompt. Le modèle lui-même n'a pas de mémoire propre entre deux appels.

Long terme

Persiste entre les sessions — nécessite un mécanisme externe (base de données ou base vectorielle, lien Module 2 et RAG du Module 4) où l'agent stocke et retrouve des faits, préférences, événements passés.

03

L'écosystème des agents aujourd'hui

Standardisation autour du Model Context Protocol.

MCP : le standard qui s'est imposé

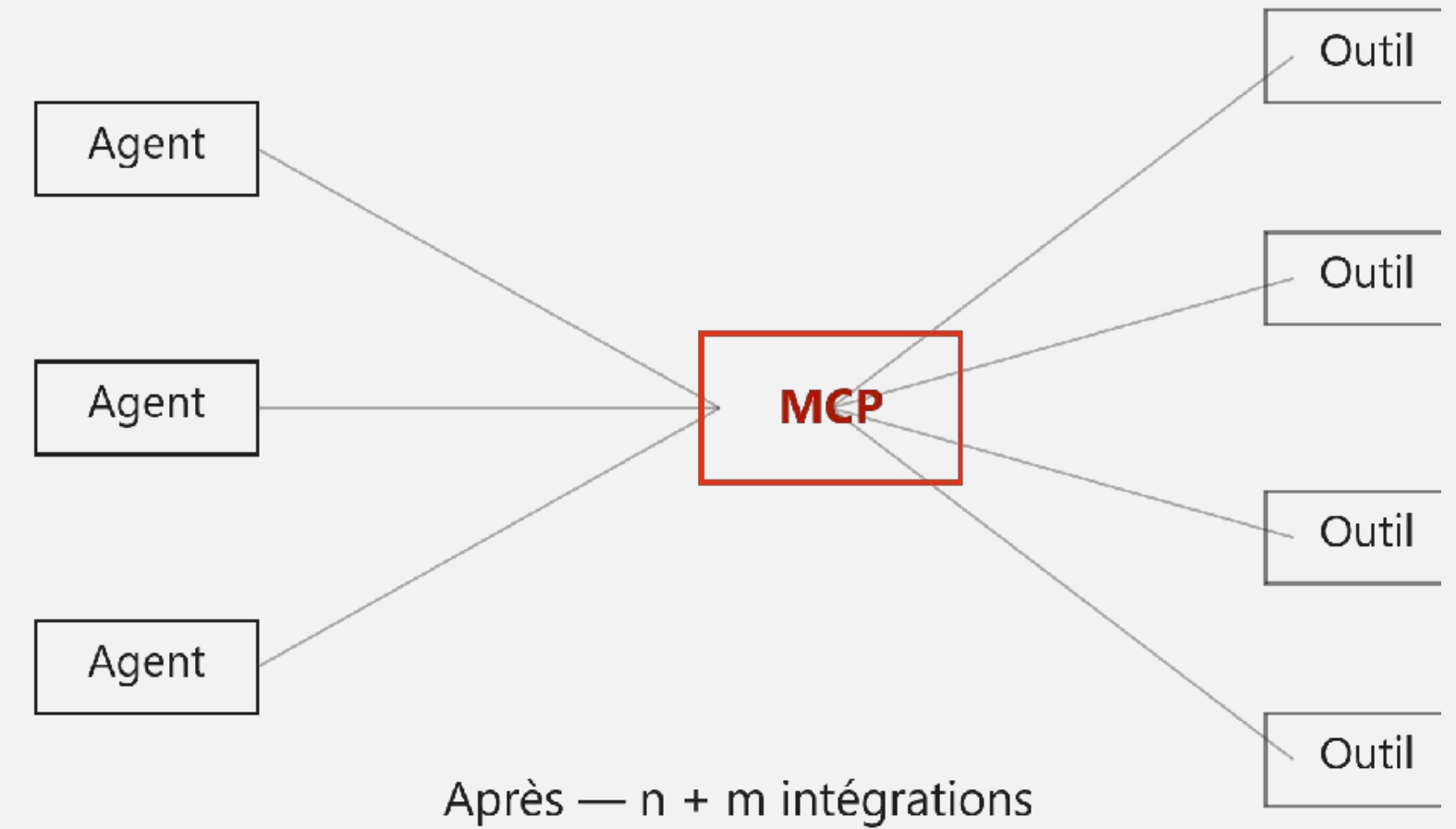
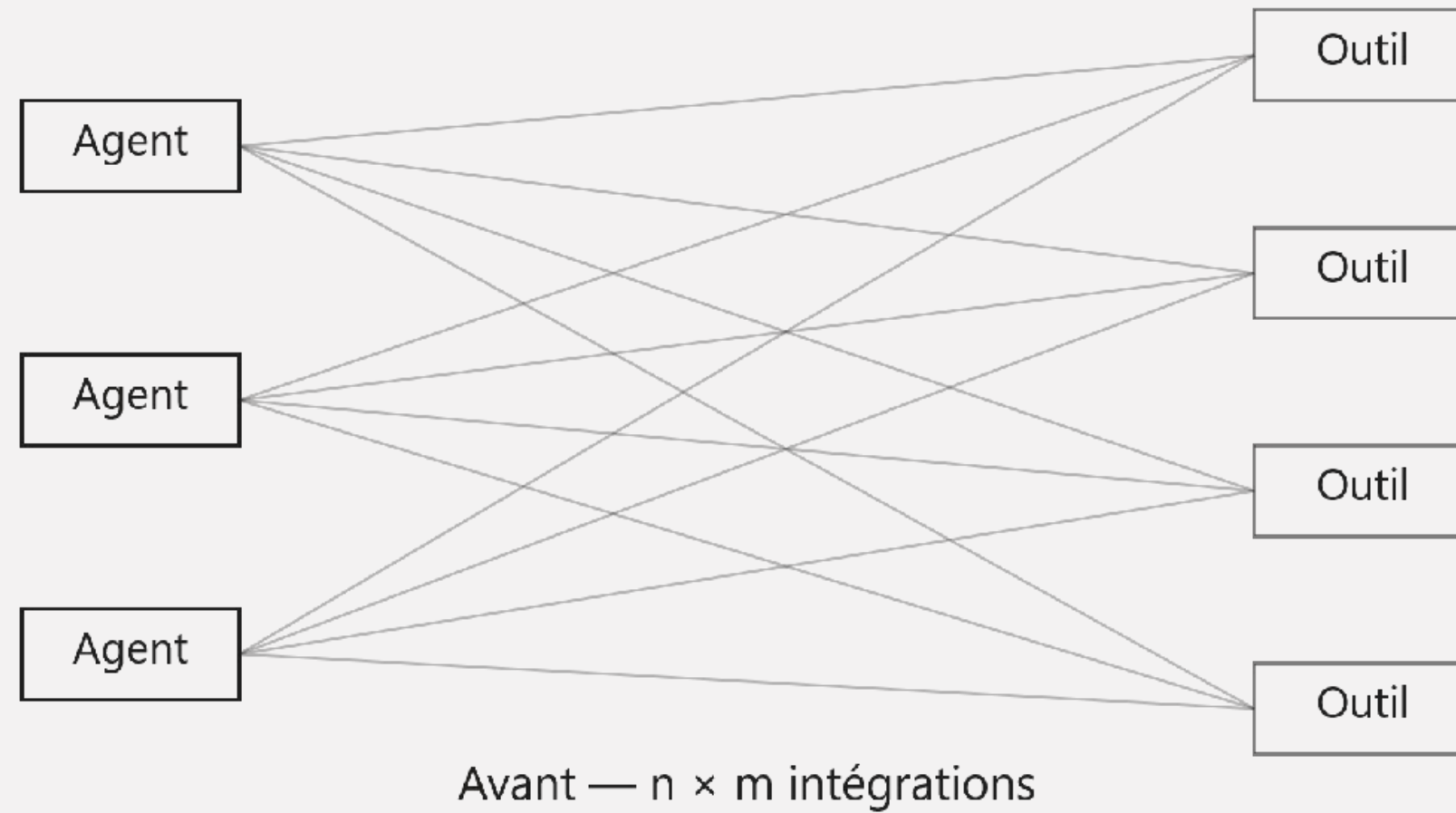
Avant 2024-2025, chaque connexion agent ↔ outil nécessitait une intégration sur-mesure — un problème d'intégration « $n \times m$ » (n modèles/agents, m outils).

Le **Model Context Protocol (MCP)** est un standard ouvert : n'importe quel client LLM compatible dialogue avec n'importe quel serveur d'outils, de ressources ou de prompts compatible — $n \times m$ devient $n+m$.

Chronologie : introduit par Anthropic en novembre 2024, puis donné en décembre 2025 à une fondation neutre — l'Agentic AI Foundation (AAIF), hébergée par la Linux Foundation, avec OpenAI et Block comme co-fondateurs, et AWS, Google, Microsoft, Cloudflare, Bloomberg comme membres platine.

Ce mouvement a rendu réelle la métaphore du « USB-C de l'IA » : un seul protocole pour connecter n'importe quel modèle à n'importe quel outil.

$n \times m$ intégrations $\rightarrow n + m$ intégrations



Le vocabulaire MCP

Tools

Des fonctions exécutables que l'agent peut déclencher.

Resources

Des données que l'agent peut lire — un peu comme des fichiers en lecture seule ou des réponses d'API.

Prompts

Des modèles de conversation standardisés qui cadrent la façon dont l'agent doit utiliser un outil ou une donnée donnée.

Une adoption rapide, pas encore mature en sécurité

L'adoption rapide et généralisée du protocole s'est accompagnée de vrais problèmes de sécurité : plus de 30 failles (CVE) référencées début 2026.

Incidents documentés : fuite de données inter-clients, faille d'accès à des fichiers, attaques par « empoisonnement d'outils » sur des serveurs open source.

La rapidité d'adoption d'un standard n'est pas toujours synonyme de maturité en sécurité — un vrai point de vigilance pour tout projet agentique en entreprise. À noter aussi : le protocole A2A (Agent-to-Agent), complémentaire à MCP, gère la communication entre plusieurs agents entre eux (MCP reste concentré sur agent ↔ outils/données).

Le concept de Skills

Un **Skill** est un ensemble structuré d'instructions, de bonnes pratiques et d'outils packagés pour une tâche récurrente — ex. « générer un document Word bien formaté », « suivre la charte éditoriale de l'entreprise ».

Rend un agent plus fiable et reproductible sur des tâches spécifiques, sans tout ré-expliquer dans chaque prompt.

C'est typiquement le genre de brique que vous manipulerez en construisant vos propres agents dans les modules pratiques à venir.

Le concept de Skills

Exemple de Skills:

<https://github.com/mattpocock/skills>

<https://github.com/mattpocock/skills/blob/main/skills/productivity/wait-what>

Synthèse

Un agent = un LLM (le « cerveau ») + une boucle de décision (ReAct) + des outils qu'il peut déclencher (tool calling) + une mémoire (court terme = contexte, long terme = stockage externe).

L'écosystème s'est standardisé très vite autour de MCP — un vrai atout pour construire rapidement, mais avec de vraies questions de sécurité encore ouvertes.

Module 1 (comment un modèle est fabriqué) → Modules 2-3 (écosystème et prix) → Module 4 (comment on l'enrichit en connaissance) → Module 5 (comment on le rend autonome).

Transition directe vers le Module 6, la synthèse finale de la journée.

Synthèse & fil conducteur des 6 mois

Ce qu'il faut retenir de la journée — et où cela mène.

Durée : 30 min · Fin d'après-midi

Objectifs du module

Faire le lien explicite entre chaque module de la journée et la carte mentale globale du programme.

Rassurer : pas besoin de tout maîtriser aujourd'hui, mais avoir une carte routière claire de ce qui attend sur six mois.

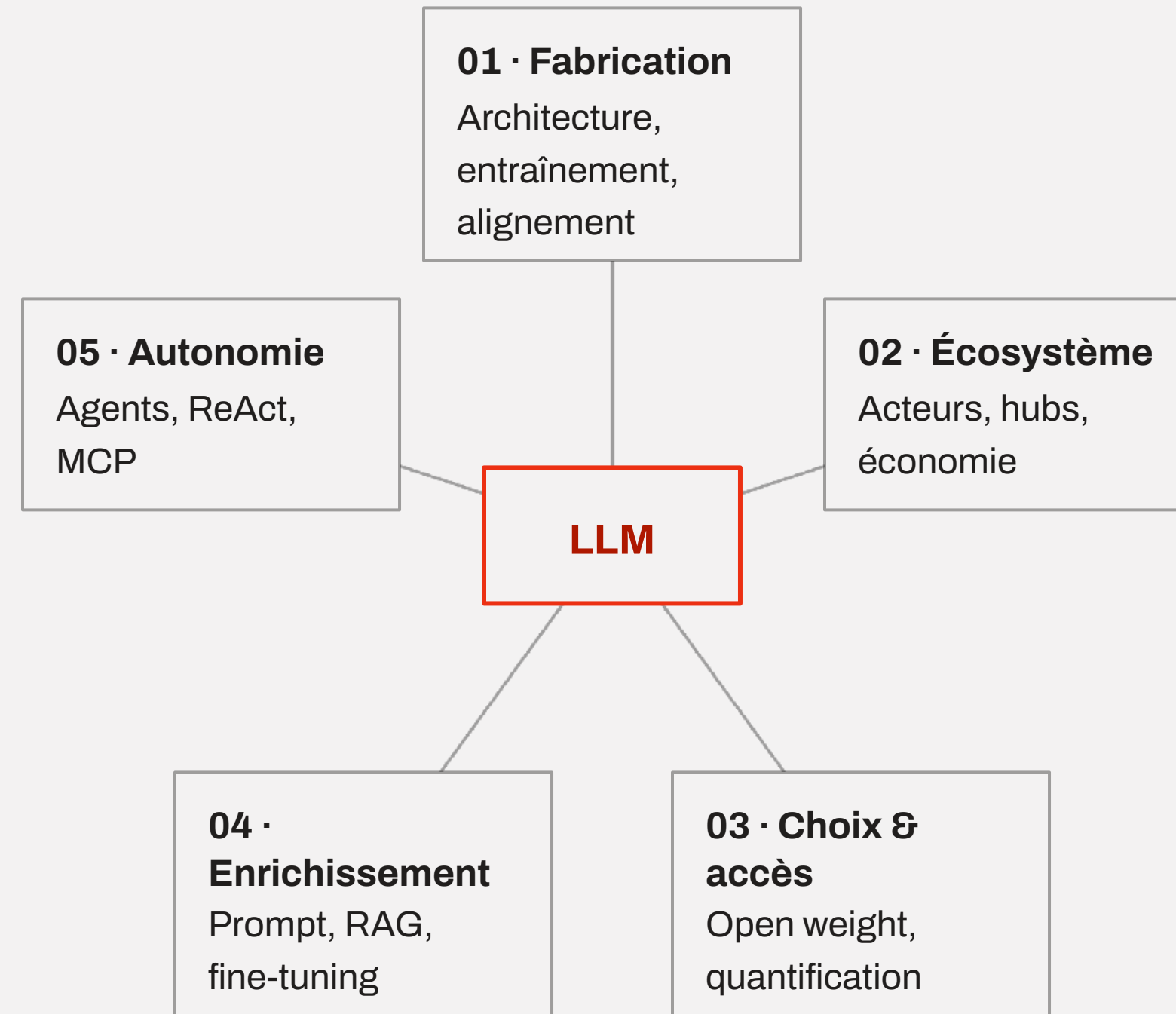
Créer de l'anticipation positive pour la suite du cursus.

01

La carte mentale de la journée

Les six modules, reliés entre eux.

Un modèle, cinq questions



02

Passerelle vers la suite du cursus

Cette journée est le prérequis conceptuel des six prochains mois.

La prochaine étape : la pratique

Cette journée est le prérequis conceptuel de tous les modules techniques approfondis à venir. Prochaine étape naturelle : un cours pratique sur vLLM / Llama — vous allez manipuler concrètement ce qui n'a été vu aujourd'hui qu'au niveau conceptuel (moteurs d'inférence du Module 2, quantification du Module 3).

Chaque branche sera rouverte en pratique

RAG

Repris en pratique avec du code Python et une vraie base vectorielle.

Agents

Repris avec la construction d'un agent réel, outillé en MCP.

Fine-tuning

Repris avec une session pratique de LoRA/QLoRA sur un modèle open weight.

Vous êtes ici



Jour 1

Aujourd'hui



Pratique vLLM

Inférence appliquée



RAG

Construction complète



Fine-tuning

LoRA / QLoRA appliqué



Projet agent

Complet, de bout en bout

Pour clôturer : le réflexe de veille

La matière vue aujourd'hui évolue vite — tout le contenu du Module 2 en particulier. Développez dès maintenant le réflexe de veille : leaderboards (Hugging Face, Artificial Analysis, LMArena), blogs techniques des labs.

Cette journée n'est pas une photographie figée du domaine — c'est une carte pour continuer à s'y repérer.